Placement DSA Masterbook

One day-wise system: Reading + Blind Typing

Placement / OA Coding

October 2026

How to use this book

This is the **single source of truth** for weekly DSA placement preparation. There is no separate Reading chapter and no separate Blind Typing chapter: each algorithm/template appears **once**, on the day you actually practice it.

Daily flow: read the day cues first, type the canonical cards from memory, then spend the final block on the day's Hards or on retyping misses. Reading is passive recall; the code below is also the typing source.

Rule	Use
One canonical code copy	Never hunt for the same template in another section.
10-30 min reading	Read only today's section; recall invariant/state before looking back.
60 min typing	Daily Core first, then today's cards, then Hards/retype.
2-4 Hards/week	Extract state/invariant, failure of the obvious idea, transition, and complexity.

Weekly map

Day	Main cluster	Reading	Typing
Monday	Arrays / Search / Prefix	10-20m	60m
Tuesday	Sliding Window / Stack / Heap	10-20m	60m
Wednesday	Linked List / Strings / Cache	10-20m	60m
Thursday	Graphs / Trees / DSU / MST	10-20m	60m
Friday	DP / Backtracking	10-20m	60m
Saturday	Fenwick / Segment Tree	10-20m	60m
Sunday	C++ / Quant / Hard review / Advanced	10-30m	60m

Weekly Index

One canonical path: open today's row, read its recall cues, then work the cards in order. TYPE active memory typing SOLVE timed transfer READ reference only.

Day	Focus	Read	Page
Monday	Arrays / Search / Prefix	10-20m	4
Tuesday	Sliding Window / Stack / Heap	10-20m	9
Wednesday	Linked List / Strings / Cache	10-20m	13
Thursday	Graphs / Trees / DSU / MST	10-20m	17
Friday	DP / Backtracking	10-20m	22
Saturday	Fenwick / Segment Tree	10-20m	25
Sunday	C++ / Quant / Hard review / Advanced Vault	10-30m	29

60-minute loop 0-8m Daily Core → 8-38m day cards → 38-53m Hard transfer → 53-60m retype misses + one edge case.

Placement-wide variant rules

Binary search
exact target → ordinary search; first $\geq x$ → `lower_bound`; first $> x$ → `upper_bound`; optimize answer → monotone `ok(mid)`.

Shortest path
unweighted → BFS; weights 0/1 → 0-1 BFS; non-negative → Dijkstra; negative edges → Bellman-Ford; all-pairs/small V → Floyd-Warshall.

Trees / DSU / pointers
DSU for connectivity/merges; LCA for ancestor queries; pointers/references require explicit lifetime + ownership reasoning.

Sliding window
longest distinct → last-seen/frequency; minimum window → required counts + satisfied counter; fixed-size maximum → monotonic deque.

DP loops
0/1 knapsack downward in capacity; unbounded upward. Strict LIS uses `lower_bound`; non-decreasing uses `upper_bound`.

Segment Trees
merge must be associative; commutativity is optional. Lazy propagation needs correct node application + ordered tag composition. Range `chmin/chmax` generally calls for Segment Tree Beats.

Daily Core - type every day

0-8 minutes every day. These two cards are intentionally not copied into every weekday section; this is the only canonical copy.

TYPE 1. Math: qexp / gcd / ext_gcd / inv_mod

```cpp
long long qexp(long long a, long long b, long long m = 1000000007LL) {
    long long res = 1; a %= m;
    while (b) { if (b & 1) res = (long long)((__int128_t)res * a % m); a = (long long)((__int128_t)a * a %
     ↪  m); b >>= 1; }
    return res;
}
long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }
long long ext_gcd(long long a, long long b, long long &x, long long &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, d = ext_gcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b); return d;
}
long long inv_mod(long long a, long long m) {
    long long x, y; long long g = ext_gcd(a, m, x, y);
    if (g != 1) return -1;
    return (m + x % m) % m;
}
```

TYPE 2. Bits: core bit operations

```cpp
#define getbit(x,i) (((x)>>(i))&1LL)            // Extract bit at index i exactly as 0 or 1
#define setbit(x,i) ((x)|(1LL<<(i)))            // Set at index i to 1
#define clearbit(x,i) ((x)&(~(1LL<<(i))))       // Set at index i to 0
#define togglebit(x,i) ((x)^(1LL<<(i)))         // Toggle between 0 and 1
#define lowbit(x) ((x)&(-(x)))                  // Isolate lowest set bit
#define strip_lowbit(x) ((x)-((x)&(-(x))))      // Clear lowest set bit
#define is_pow2(x) ((x)&&(((x)&((x)-1LL))==0))  // Check if exact power of 2
#define is_allones(x) ((x)&&(((x)&((x)+1LL))==0)) // Check if binary representation is all 1s
inline unsigned long long all_ones(int n) { return n <= 0 ? 0ULL : n >= 64 ? ~0ULL : (1ULL << n) - 1ULL; } //
 ↪  Generate n-bit mask safely

// Built-ins: __builtin_popcountll(x), __builtin_ctzll(x) [trailing 0s], __builtin_clzll(x) [leading 0s]

// Count 1s in binary representations of all numbers from 1 to N in O(log N).
// __int128 is used because the total count can exceed 64-bit even when N fits in long long.
__int128 count1s_ton(long long n) {
    __int128 c = 0;
    for(int i=62; i>=0; i--) {
        if(n & (1LL<<i)) {
            c += (i ? (__int128(1)<<(i-1))*i : 0) + (n-(1LL<<i)+1);
            n -= (1LL<<i);
        }
    }
    return c;
}
```

Monday

Focus: Arrays / Search / Prefix / Compression

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min cards; 38-53 min Hard transfer; 53-60 min retype misses + one edge case. **Week A:** Two Sum -> 3Sum -> bounds -> answer-BS. **Week B:** Kadane -> rotated search -> intervals -> prefix/compression.

Read / recall first

- **Prefix vs difference:** prefixes answer repeated ranges; differences mark repeated range updates.
- **Binary-search-on-answer:** the hard part is the monotone predicate, not the loop.
- **Coordinate compression:** preserve order/rank, not the original numeric gaps.
- **Two-pointer after sorting:** use the sorted invariant to eliminate a dimension of search.

Canonical cards - one copy: read once, then type/solve as marked

TYPE **1. Two Sum**

Problem: Find two array elements whose values add to the target.

```cpp
// Hash complements; for duplicates, the stored index may be reused only after checking the current element.
vector<int> two_sum(const vector<int>& a, int target) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = pos.find(target - a[i]);
        if (it != pos.end()) return {it->second, i};
        pos[a[i]] = i;
    }
    return {};
}
```

TYPE **2. 3Sum**

Problem: Find all unique triples whose values sum to zero.

```cpp
// Sorting removes duplicate triples cleanly and reduces the inner search to O(n).
vector<vector<int>> three_sum(vector<int> a) {
    sort(a.begin(), a.end());
    vector<vector<int>> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (i && a[i] == a[i-1]) continue;
        int l = i + 1, r = (int)a.size() - 1;
        while (l < r) {
            long long s = 1LL * a[i] + a[l] + a[r];
            if (s < 0) ++l;
            else if (s > 0) --r;
            else {
                ans.push_back({a[i], a[l], a[r]});
                int x = a[l], y = a[r];
                while (l < r && a[l] == x) ++l;
                while (l < r && a[r] == y) --r;
            }
        }
    }
    return ans;
}
```

TYPE 3. First/last occurrence and lower/upper bound

```cpp
// These helpers return iterator positions; they work on sorted arrays and handle duplicates.
int lower_bound_idx(const vector<int>& a, int x) {
    int l = 0, r = a.size();
    while (l < r) { int m = l + (r-l)/2; if (a[m] < x) l = m + 1; else r = m; }
    return l;
}
int upper_bound_idx(const vector<int>& a, int x) {
    int l = 0, r = a.size();
    while (l < r) { int m = l + (r-l)/2; if (a[m] <= x) l = m + 1; else r = m; }
    return l;
}
```

TYPE 4. Aggressive cows / maximize minimum distance

Problem: Place cows in stalls so the minimum pairwise distance is maximized.

```cpp
// Greedy placement is optimal for a fixed distance; binary search finds the largest feasible distance.
bool can_place(const vector<int>& x, int k, int d) {
    int cnt = 1, last = x[0];
    for (int i = 1; i < (int)x.size(); ++i) if (x[i] - last >= d) last = x[i], ++cnt;
    return cnt >= k;
}
int aggressive_cows(vector<int> x, int k) {
    sort(x.begin(), x.end());
    int l = 0, r = x.back() - x.front(), ans = 0;
    while (l <= r) {
        int m = l + (r-l)/2;
        if (can_place(x, k, m)) ans = m, l = m + 1;
        else r = m - 1;
    }
    return ans;
}
```

TYPE 5. Maximum subarray / Kadane

Problem: Find the maximum sum of a contiguous subarray.

```cpp
// Start with a[0], not zero, so an array of all negatives returns its true maximum.
long long kadane(const vector<long long>& a) {
    if (a.empty()) return 0;
    long long cur = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        cur = max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

TYPE 6. Search in rotated sorted array

Problem: Search for a target in a sorted array rotated at an unknown pivot.

```cpp
// Distinguish the sorted half and test whether the target lies inside it.
int search_rotated(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] == target) return m;
        if (a[l] <= a[m]) {
            if (a[l] <= target && target < a[m]) r = m - 1;
            else l = m + 1;
        } else {
            if (a[m] < target && target <= a[r]) l = m + 1;
            else r = m - 1;
        }
    }
    return -1;
}
```

TYPE 7. Merge intervals

Problem: Merge all overlapping intervals into disjoint ranges.

```cpp
// Sorting by start guarantees every overlapping interval appears next to the current merged interval.
vector<vector<int>> merge_intervals(vector<vector<int>> a) {
    if (a.empty()) return {};
    sort(a.begin(), a.end());
    vector<vector<int>> ans{a[0]};
    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i][0] <= ans.back()[1]) ans.back()[1] = max(ans.back()[1], a[i][1]);
        else ans.push_back(a[i]);
    }
    return ans;
}
```

TYPE 8. Prefix Sum / Prefix XOR / Difference Array / 2D Prefix

```cpp
// Prefix arrays turn repeated range arithmetic into O(1) queries; difference arrays turn range updates into
↪ O(1) marks.
vector<long long> prefix(const vector<long long>& a) {
    int n = a.size(); vector<long long> p(n + 1);
    for (int i = 0; i < n; ++i) p[i + 1] = p[i] + a[i];
    return p;
}
vector<long long> prefix_xor(const vector<long long>& a) {
    int n = a.size(); vector<long long> p(n + 1);
    for (int i = 0; i < n; ++i) p[i + 1] = p[i] ^ a[i];
    return p;
}
vector<long long> build_diff(int n, const vector<array<int,3>>& ops) {
    vector<long long> d(n + 1);
    for (auto [l, r, x] : ops) d[l] += x, d[r + 1] -= x;
    for (int i = 1; i < n; ++i) d[i] += d[i - 1];
    d.resize(n); return d;
}
vector<vector<long long>> prefix2d(const vector<vector<long long>>& a) {
    int m = a.size(), n = m ? a[0].size() : 0;
    vector<vector<long long>> p(m + 1, vector<long long>(n + 1));
    for (int i = 0; i < m; ++i) for (int j = 0; j < n; ++j)
        p[i + 1][j + 1] = a[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j];
    return p;
}
long long rect_sum(const vector<vector<long long>>& p, int r1, int c1, int r2, int c2) {
```

```
    return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
}
```

TYPE 9. Coordinate Compression

```
// Compress only when ordering/rank matters; preserve the original values separately if they are needed later.
vector<long long> compress_values(vector<long long> a) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    return a;
}
int compressed_id(const vector<long long>& xs, long long x) {
    return lower_bound(xs.begin(), xs.end(), x) - xs.begin();
}
```

TYPE 10. Hard transfer - 3165 - Four-state Segment Tree core

Problem: Practice the four-state Segment Tree core used by 3165; the query-processing/modulo wrapper is intentionally omitted. **Link:** LeetCode problem 3165

```
// Node state encodes whether the segment's left/right endpoints are selected; merge rejects adjacent
 ↪  selections across the boundary.
struct NonAdjSeg {
    struct Node { long long s[2][2]{}; };
    int n; vector<Node> st;
    NonAdjSeg(const vector<int>& a) : n(a.size()), st(4 * n) { build(1, 0, n - 1, a); }
    Node merge(const Node& A, const Node& B) {
        Node C;
        for (int l = 0; l < 2; ++l) for (int r = 0; r < 2; ++r) C.s[l][r] = LLONG_MIN / 4;
        for (int l = 0; l < 2; ++l) for (int x = 0; x < 2; ++x)
            for (int y = 0; y < 2; ++y) for (int r = 0; r < 2; ++r)
                if (!(x && y)) C.s[l][r] = max(C.s[l][r], A.s[l][x] + B.s[y][r]);
        return C;
    }
    void build(int v, int l, int r, const vector<int>& a) {
        if (l == r) { st[v].s[0][0] = st[v].s[0][1] = st[v].s[1][0] = 0; st[v].s[1][1] = max(0, a[l]); return;
         ↪  }
        int m = (l + r) / 2; build(v * 2, l, m, a); build(v * 2 + 1, m + 1, r, a); st[v] = merge(st[v * 2],
         ↪  st[v * 2 + 1]);
    }
    void update(int v, int l, int r, int p, int x) {
        if (l == r) { st[v] = Node{}; st[v].s[1][1] = max(0, x); return; }
        int m = (l + r) / 2; if (p <= m) update(v * 2, l, m, p, x); else update(v * 2 + 1, m + 1, r, p, x);
         ↪  st[v] = merge(st[v * 2], st[v * 2 + 1]);
    }
    void update(int p, int x) { update(1, 0, n - 1, p, x); }
    long long answer() const { return st[1].s[1][1]; }
};
```

SOLVE 11. Hard transfer - 3139 - Minimum Cost to Equalize Array

Problem: Find the minimum cost to make all array values equal using single-element and paired operations. **Link:** LeetCode problem

READ **Next permutation**

```cpp
// Find the rightmost ascent, swap with the rightmost larger value, then reverse the suffix.
void next_perm(vector<int>& a) {
    int n = a.size(), i = n - 2;
    while (i >= 0 && a[i] >= a[i+1]) --i;
    if (i < 0) { reverse(a.begin(), a.end()); return; }
    int j = n - 1;
    while (a[j] <= a[i]) --j;
    swap(a[i], a[j]);
    reverse(a.begin() + i + 1, a.end());
}
```

READ **Trapping Rain Water**

```cpp
// Move the side with the smaller boundary because its trapped water is already determined by that side.
long long trap_water(const vector<int>& h) {
    int l = 0, r = (int)h.size() - 1;
    long long lm = 0, rm = 0, ans = 0;
    while (l <= r) {
        if (h[l] <= h[r]) {
            lm = max(lm, (long long)h[l]);
            ans += lm - h[l++];
        } else {
            rm = max(rm, (long long)h[r]);
            ans += rm - h[r--];
        }
    }
    return ans;
}
```

READ **Binary search on the answer**

Remember: Turn an optimization objective into a monotone feasibility test; search the boundary of feasible values.

```cpp
template<class F>
long long first_true(long long lo, long long hi, F ok) {
    long long ans = hi;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (ok(mid)) ans = mid, hi = mid - 1;
        else lo = mid + 1;
    }
    return ans;
}
```

Tuesday

Focus: Sliding Window / Stack / Heap

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min cards; 38-53 min Hard transfer; 53-60 min retype misses + one edge case. **Week A:** unique -> min-window -> sliding max -> NGE. **Week B:** Top-K -> min stack -> histogram -> two heaps.

Read / recall first

- **Window invariant:** decide what must be true for the current `[l,r]`, then move only the boundary that restores it.
- **Monotonic deque:** indices expire from the front; dominated candidates disappear from the back.
- **Monotonic stack:** maintain the exact relation the next-greater/smaller query needs.
- **Heap patterns:** top-K uses bounded heap size; running median uses two heaps with a size invariant.

Hard rotation: **3245 Alternating Groups III** - *Stress.* Maintain dynamic circular alternating runs; order-statistic/Fenwick thinking is the trigger.; **3962 Maximum Subarray Sum After at Most K Swaps** - *Stress.* Think in terms of the best achievable segment plus controlled rearrangement/order statistics.

Canonical cards - one copy: read once, then type/solve as marked

TYPE **1. Longest substring without repeating characters**

Problem: Find the longest substring containing no repeated character.

```
// Move the left boundary past the previous occurrence so every active character stays unique.
int longest_unique(const string& s) {
    vector<int> last(256,-1); int l=0,ans=0;
    for(int r=0;r<(int)s.size();++r){
        l=max(l,last[(unsigned char)s[r]]+1); last[(unsigned char)s[r]]=r;
        ans=max(ans,r-l+1);
    }
    return ans;
}
```

TYPE **2. Minimum window substring**

Problem: Find the shortest substring containing all characters of a target multiset.

```
// Expand until all required counts are satisfied, then shrink greedily from the left.
string min_window(const string& s,const string& t){
    vector<int> need(256), have(256); for(char c:t) ++need[(unsigned char)c];
    int missing=t.size(), l=0,best=INT_MAX,beg=0;
    for(int r=0;r<(int)s.size();++r){
        unsigned char c=s[r]; if(need[c]&&have[c]++<need[c]) --missing;
        while(!missing){
            if(r-l+1<best) best=r-l+1,beg=l;
            unsigned char d=s[l++]; if(need[d]&&have[d]--<=need[d]) ++missing;
        }
    }
    return best==INT_MAX?string():s.substr(beg,best);
}
```

TYPE 3. Sliding window maximum

Problem: Return the maximum value in every fixed-size sliding window.

```cpp
// Expire left indices first, then remove smaller values from the back so the front is always the maximum.
vector<int> sliding_max(const vector<int>& a, int k) {
    deque<int> dq; vector<int> ans;
    for (int i=0;i<(int)a.size();++i) {
        while (!dq.empty() && dq.front() <= i-k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k-1) ans.push_back(a[dq.front()]);
    }
    return ans;
}
```

TYPE 4. Next greater element

Problem: For each element, find the first greater value to its right.

```cpp
// Pop while the current value is larger; every popped index gets its first greater element.
vector<int> next_greater(const vector<int>& a) {
    vector<int> ans(a.size(),-1); vector<int> st;
    for(int i=0;i<(int)a.size();++i){
        while(!st.empty() && a[st.back()]<a[i]) ans[st.back()]=a[i],st.pop_back();
        st.push_back(i);
    }
    return ans;
}
```

TYPE 5. Top K frequent elements

Problem: Return the k values with the highest frequencies.

```cpp
// Keep only k candidates in the heap; the smallest frequency is the one discarded first.
vector<int> top_k_frequent(const vector<int>& a, int k) {
    unordered_map<int,int> f;
    for (int x:a) ++f[x];
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    for (auto [x,c]:f) {
        pq.push({c,x});
        if ((int)pq.size()>k) pq.pop();
    }
    vector<int> ans;
    while(!pq.empty()) ans.push_back(pq.top().second), pq.pop();
    return ans;
}
```

TYPE 6. Min stack

```cpp
// Empty-stack behavior is left to the caller; use LLONG_MAX as an identity only when appropriate.
struct MinStack {
    vector<pair<int,int>> st;
    void push(int x){ int mn=st.empty()?x:min(x,st.back().second); st.push_back({x,mn}); }
    void pop(){ st.pop_back(); }
    int top() const { return st.back().first; }
    int getMin() const { return st.back().second; }
};
```

TYPE 7. Largest Rectangle in Histogram

Problem: Find the largest rectangle that can be formed under histogram bars.

```cpp
// Maintain increasing heights; when a smaller bar arrives, every popped bar has now found its right boundary.
long long largestRectangleArea(const vector<int>& h) {
    int n = h.size(); long long ans = 0; vector<int> st;
    for (int i = 0; i <= n; ++i) {
        int cur = (i == n ? 0 : h[i]);
        while (!st.empty() && h[st.back()] > cur) {
            int ht = h[st.back()]; st.pop_back();
            int left = st.empty() ? -1 : st.back();
            ans = max(ans, 1LL * ht * (i - left - 1));
        }
        st.push_back(i);
    }
    return ans;
}
```

TYPE 8. Two Heaps / Running Median

Problem: Maintain the median while numbers arrive online.

```cpp
// Keep left as a max-heap and right as a min-heap; rebalance so their sizes differ by at most one.
struct MedianDS {
    priority_queue<int> lo;
    priority_queue<int, vector<int>, greater<int>> hi;
    void add(int x) {
        if (lo.empty() || x <= lo.top()) lo.push(x); else hi.push(x);
        if (lo.size() > hi.size() + 1) hi.push(lo.top()), lo.pop();
        if (hi.size() > lo.size()) lo.push(hi.top()), hi.pop();
    }
    double median() const { return lo.size() == hi.size() ? (lo.top() + hi.top()) / 2.0 : lo.top(); }
};
```

TYPE 9. Hard transfer - 3357 - Minimize the Maximum Adjacent Element Difference

Problem: Replace unknown values so the maximum adjacent difference is as small as possible. **Link:** LeetCode problem

```cpp
// Binary-search the answer; for a candidate m, only the values adjacent to -1-runs matter, and the two
↪  replacement values are mn+m and mx-m.
class Solution {
    bool check(const vector<int>& a, int m, int x, int y) {
        int gap = 0, prev = -1;
        for (int v : a) {
            if (v == -1) { ++gap; continue; }
            if (prev != -1 && gap) {
                if (gap == 1) {
                    if (min(max(abs(prev-x), abs(v-x)), max(abs(prev-y), abs(v-y))) > m) return false;
                } else {
                    int a1 = max(abs(prev-x), abs(v-x));
                    int a2 = max(abs(prev-y), abs(v-y));
                    int a3 = max({abs(prev-x), abs(x-y), abs(v-y)});
                    int a4 = max({abs(prev-y), abs(x-y), abs(v-x)});
                    if (min({a1,a2,a3,a4}) > m) return false;
                }
            }
            prev = v; gap = 0;
        }
        auto first = [&](int st, int step) { int i = st; while (0 <= i && i < (int)a.size() && a[i] == -1) i
        ↪  += step; return (0 <= i && i < (int)a.size()) ? a[i] : -1; };
        if (a.front() == -1) { int v = first(0, 1); if (v != -1 && min(abs(v-x), abs(v-y)) > m) return false;
        ↪  }
        if (a.back() == -1) { int v = first((int)a.size()-1, -1); if (v != -1 && min(abs(v-x), abs(v-y)) > m)
        ↪  return false; }
        return true;
```

```
    }
public:
    int minDifference(vector<int>& nums) {
        int base = 0, mn = 1e9, mx = 0;
        for (int i = 1; i < (int)nums.size(); ++i) {
            if ((nums[i-1] == -1) != (nums[i] == -1)) { int v = max(nums[i-1], nums[i]); mn = min(mn, v); mx
             ↪  = max(mx, v); }
            else if (nums[i] != -1) base = max(base, abs(nums[i] - nums[i-1]));
        }
        int l = base, r = (mx - mn + 1) / 2;
        while (l < r) { int m = (l + r) / 2; if (check(nums, m, mn + m, mx - m)) r = m; else l = m + 1; }
        return l;
    }
};
```

SOLVE **10. Hard transfer - 3266 - Final Array State II**

Problem: Repeatedly multiply the current minimum element, then return the final array after a huge number of operations.
Link: LeetCode problem

Wednesday

Focus: Linked List / Strings / Cache

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min cards; 38-53 min Hard transfer; 53-60 min retype misses + one edge case. **Week A:** list core -> KMP -> Z -> LRU. **Week B:** Trie -> anagrams -> pattern matching -> cache.

Read / recall first

- **Linked lists:** pointer rewiring order matters more than syntax; protect the next pointer before cutting links.
- **KMP / Z:** preprocess reusable prefix/suffix structure so mismatches do not restart from scratch.
- **LRU:** hash map gives direct lookup; doubly linked list gives O(1) recency updates.
- **Trie:** each edge encodes one character; terminal state distinguishes a stored word from a prefix.

Hard rotation: **3504 Longest Palindrome After Substring Concatenation II** - *Stress.* Two-string palindrome DP with boundary matching.; **3463 Check If Digits Are Equal in String After Operations II** - *Stress.* Reduce the combinatorics modulo 10 via mod 2/mod 5 and CRT.; **3563 Lexicographically Smallest String After Adjacent Removals** - *Stress.* Structured removals lead to interval-style DP.; **3213 Construct String With Minimum Cost** - *Solved benchmark.* DP by prefix position; use hashing/Aho-Corasick to discover dictionary words ending at each state.

Canonical cards - one copy: read once, then type/solve as marked

TYPE **1. Linked List Core**

Problem: Core pointer operations covering reversal and merging of linked lists.

```
// Dummy nodes remove head special-cases; slow/fast gives the middle and cycle patterns with O(1) extra space.
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *cur = head;
    while (cur) { ListNode* nxt = cur->next; cur->next = prev; prev = cur; cur = nxt; }
    return prev;
}
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0), *cur = &dummy;
    while (a && b) { if (a->val <= b->val) cur->next = a, a = a->next; else cur->next = b, b = b->next; cur =
    ↪  cur->next; }
    cur->next = a ? a : b; return dummy.next;
}
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0); dummy.next = head; ListNode *slow = &dummy, *fast = &dummy;
    while (n--) fast = fast->next;
    while (fast->next) slow = slow->next, fast = fast->next;
    ListNode* del = slow->next; slow->next = del->next; delete del; return dummy.next;
}
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; if (slow == fast) return true; }
    return false;
}
```

TYPE 2. KMP prefix function / pattern matching

```cpp
// pi[i] is the longest proper prefix of s[0..i] that is also a suffix; pi drives mismatch jumps.
vector<int> prefix_function(const string& s){
    vector<int> pi(s.size());
    for(int i=1;i<(int)s.size();++i){
        int j=pi[i-1];
        while(j&&s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) ++j;
        pi[i]=j;
    }
    return pi;
}
vector<int> kmp_find(const string& text,const string& pat){
    if(pat.empty()) return {};
    string s=pat+"#"+text; auto pi=prefix_function(s); vector<int> ans;
    for(int i=(int)pat.size()+1;i<(int)s.size();++i) if(pi[i]==(int)pat.size()) ans.push_back(i-2*pat.size());
    return ans;
}
```

TYPE 3. Z algorithm

```cpp
// Maintain the rightmost Z-box and reuse mirrored information while inside the box.
vector<int> z_function(const string& s){
    int n=s.size(); vector<int> z(n);
    for(int i=1,l=0,r=0;i<n;++i){
        if(i<=r) z[i]=min(r-i+1,z[i-l]);
        while(i+z[i]<n&&s[z[i]]==s[i+z[i]]) ++z[i];
        if(i+z[i]-1>r) l=i,r=i+z[i]-1;
    }
    return z;
}
```

TYPE 4. LRU Cache

Problem: Support get/put in O(1) while evicting the least-recently-used entry.

```cpp
// Front is most recent and back is least recent; map stores node iterators for direct removal.
class LRUCache {
    int cap; list<pair<int,int>> q; unordered_map<int,list<pair<int,int>>::iterator> mp;
    void touch(list<pair<int,int>>::iterator it) { auto p=*it; q.erase(it); q.push_front(p);
     ↪ mp[p.first]=q.begin(); }
public:
    LRUCache(int c):cap(c){}
    int get(int k) {
        auto it=mp.find(k); if(it==mp.end()) return -1;
        int v=it->second->second; touch(it->second); return v;
    }
    void put(int k,int v) {
        if(mp.count(k)){ q.erase(mp[k]); mp.erase(k); }
        q.push_front({k,v}); mp[k]=q.begin();
        if((int)q.size()>cap){ mp.erase(q.back().first); q.pop_back(); }
    }
};
```

TYPE 5. Trie

```cpp
// Standard String Trie
struct Trie {
    vector<vector<int>> trie; vector<int> cnt; int num = 0;
    Trie(int max_nodes) : trie(max_nodes, vector<int>(26, 0)), cnt(max_nodes, 0) {}
    void insert(string s) {
        int u = 0;
        for (char c : s) { if (!trie[u][c - 'a']) trie[u][c - 'a'] = ++num; u = trie[u][c - 'a']; }
        cnt[u]++;
    }
};

// Bitwise 0-1 Trie (Maximum XOR queries)
struct BitTrie {
    vector<vector<int>> trie; int num;
    BitTrie(int max_el) : trie(max_el * 32, vector<int>(2, 0)), num(1) {}
    void insert(int x) {
        int u = 1;
        for (int i = 30; i >= 0; --i) {
            int bit = (x >> i) & 1;
            if (!trie[u][bit]) trie[u][bit] = ++num;
            u = trie[u][bit];
        }
    }
    int get_max_xor(int x) {
        int u = 1, ans = 0;
        for (int i = 30; i >= 0; --i) {
            int bit = (x >> i) & 1;
            if (trie[u][bit ^ 1]) { ans |= (1 << i); u = trie[u][bit ^ 1]; }
            else if (trie[u][bit]) { u = trie[u][bit]; }
            else break;
        } return ans;
    }
};
```

TYPE 6. Group anagrams

Problem: Group strings that contain the same multiset of characters.

```cpp
// Sorting the letters gives identical keys for anagrams; replace with frequency keys when n is large.
vector<vector<string>> group_anagrams(const vector<string>& s){
    unordered_map<string,vector<string>> mp;
    for(auto x:s){ string k=x; sort(k.begin(),k.end()); mp[k].push_back(x); }
    vector<vector<string>> ans; for(auto &p:mp) ans.push_back(move(p.second)); return ans;
}
```

TYPE 7. Hard transfer - 420 - Strong Password Checker

Problem: Find the minimum edits needed to make a password satisfy length, character-class, and repetition rules. **Link:** LeetCode problem

```cpp
class Solution {
public:
    int strongPasswordChecker(string password) {
        int n = password.size(), missing = 3;
        bool lo = 0, up = 0, dig = 0;
        for (char c : password) lo |= islower(c), up |= isupper(c), dig |= isdigit(c);
        missing -= lo + up + dig;
        int replaces = 0, one = 0, two = 0;
        for (int i = 2; i < n;) {
            if (password[i] == password[i-1] && password[i] == password[i-2]) {
                int len = 2;
                while (i < n && password[i] == password[i-1]) ++len, ++i;
                replaces += len / 3;
                if (len % 3 == 0) ++one;
                if (len % 3 == 1) ++two;
            } else ++i;
        }
        if (n < 6) return max(6 - n, missing);
        if (n <= 20) return max(replaces, missing);
        int del = n - 20;
        replaces -= min(one, del);
        replaces -= min(max(del - one, 0), 2 * two) / 2;
        replaces -= max(del - one - 2 * two, 0) / 3;
        return del + max(replaces, missing);
    }
};
```

SOLVE 8. Hard transfer - 3303 - First Almost Equal Substring

Problem: Find the first substring that differs from a pattern in at most one position. **Link:** LeetCode problem

Thursday

Focus: Graphs / Trees / DSU / MST

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min cards; 38-53 min Hard transfer; 53-60 min retype misses + one edge case. **Week A:** BFS/DFS -> grid -> Dijkstra -> Kahn. **Week B:** DSU -> LCA -> Kruskal -> diameter -> BST.

Read / recall first

- **Graph choice:** BFS/DFS for unweighted reachability, 0-1 BFS for binary weights, Dijkstra for non-negative weighted paths.
- **Kahn:** indegree zero is the invariant; processed count detects a cycle.
- **DSU:** parent + size/rank compresses connectivity operations to near-constant amortized time.
- **Tree templates:** binary lifting stores powers-of-two ancestors; diameter is two farthest traversals; Kruskal sorts edges and keeps only DSU-safe ones.

Hard rotation: **329 Longest Increasing Path in a Matrix** - *Stress.* Strictly increasing grid edges form a DAG; memoize each cell.; **3486 Longest Special Path II** - *Stress.* Tree DFS + root-to-node sliding window + prefix distances; focus on bookkeeping.

Canonical cards - one copy: read once, then type/solve as marked

TYPE 1. BFS / DFS graph traversal

```cpp
// BFS gives minimum edge count in an unweighted graph; DFS is the usual component traversal.
vector<int> bfs_dist(int s,const vector<vector<int>>& g){
    vector<int> d(g.size(),-1);queue<int>q;q.push(s);d[s]=0;
    while(!q.empty()){int u=q.front();q.pop();for(int v:g[u])if(d[v]<0)d[v]=d[u]+1,q.push(v);}
    return d;
}
```

TYPE 2. Grid DFS/BFS Scaffold

```cpp
// Four-direction traversal is the reusable core for islands, flood fill, rotten oranges, shortest grids, and
↪  increasing-path variants.
vector<pair<int,int>> dirs = {{1,0},{-1,0},{0,1},{0,-1}};
void grid_dfs(int r, int c, const vector<vector<int>>& g, vector<vector<char>>& vis) {
    int m = g.size(), n = g[0].size(); vis[r][c] = 1;
    for (auto [dr, dc] : dirs) { int nr = r + dr, nc = c + dc; if (0 <= nr && nr < m && 0 <= nc && nc < n &&
    ↪  !vis[nr][nc]) grid_dfs(nr, nc, g, vis); }
}
```

TYPE 3. Dijkstra shortest path

```cpp
// The stale-pair check is mandatory with a standard priority_queue because decrease-key is not implemented.
vector<long long> dijkstra(int s,const vector<vector<pair<int,int>>>& g){
    int n=g.size();vector<long long>d(n,4'000'000'000'000'000'000LL);priority_queue<pair<long
    ↪  long,int>,vector<pair<long long,int>>,greater<pair<long long,int>>>pq;
    d[s]=0;pq.push({0,s});
    while(!pq.empty()){auto [du,u]=pq.top();pq.pop();if(du!=d[u])continue;for(auto
    ↪  [v,w]:g[u])if(d[v]>du+w)d[v]=du+w,pq.push({d[v],v});}
    return d;
}
```

TYPE 4. Topological sort (Kahn)

```
// A DAG has exactly n vertices removed; fewer means a cycle prevented full ordering.
vector<int> topo_sort(const vector<vector<int>>& g){
    int n=g.size();vector<int>in(n);for(int u=0;u<n;++u)for(int v:g[u])++in[v];
    queue<int>q;for(int i=0;i<n;++i)if(!in[i])q.push(i);vector<int>ans;
    while(!q.empty()){int u=q.front();q.pop();ans.push_back(u);for(int v:g[u])if(!--in[v])q.push(v);}
    if((int)ans.size()!=n)return {};return ans;
}
```

TYPE 5. Disjoint Set Union

```
struct DSU {
    vector<int> p, sz; int comps;
    DSU(int n) : comps(n), p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int u) { return u == p[u] ? u : p[u] = find(p[u]); }
    bool merge(int u, int v) {
        u = find(u), v = find(v); if (u == v) return false;
        if (sz[u] > sz[v]) swap(u, v);
        p[u] = v; sz[v] += sz[u]; comps--; return true;
    }
};
```

TYPE 6. LCA via binary lifting

```
// Here the vertices are stored in 1-based indexing (so `0` can be used as parent for root)
struct LCA {
    int n, l; vector<vector<int>> up, adj; vector<int> dep;
    LCA(int n) : n(n), l(ceil(log2(n))), up(n + 1, vector<int>(l + 1)), dep(n + 1), adj(n + 1) {}
    void dfs(int u, int p = 0, int d = 0) {
        up[u][0] = p; dep[u] = d;
        for (int i = 1; i <= l; i++) up[u][i] = up[up[u][i - 1]][i - 1];
        for (int v : adj[u]) if (v != p) dfs(v, u, d + 1);
    }
    int lca(int u, int v) {
        if (dep[u] < dep[v]) swap(u, v);
        for (int i = l; i >= 0; i--) if (dep[u] - (1 << i) >= dep[v]) u = up[u][i];
        if (u == v) return u;
        for (int i = l; i >= 0; i--) if (up[u][i] != up[v][i]) u = up[u][i], v = up[v][i];
        return up[u][0];
    }
};
```

TYPE 7. Kruskal MST

```
// The first n-1 edges that connect different components form the MST when the graph is connected.
struct Edge{int u,v;long long w;};
long long kruskal(int n,vector<Edge> e){
    sort(e.begin(),e.end(),[](auto&a,auto&b){return a.w<b.w;});DSU d(n);long long ans=0;int used=0;
    for(auto x:e)if(d.merge(x.u,x.v)){ans+=x.w;if(++used==n-1)break;}
    return used==n-1?ans:LLONG_MAX;
}
```

TYPE 8. Tree diameter

Problem: Find the longest path between any two nodes of a tree.

```cpp
// Diameter endpoints are farthest from one another in a tree; weighted variants replace BFS distance with
 ↪ DFS/Dijkstra.
pair<int,int> farthest(int s,const vector<vector<int>>&g){
    queue<int>q;q.push(s);vector<int>d(g.size(),-1);d[s]=0;int best=s;
    while(!q.empty()){int u=q.front();q.pop();for(int
     ↪ v:g[u])if(d[v]<0)d[v]=d[u]+1,q.push(v),best=d[v]>d[best]?v:best;}
    return {best,d[best]};
}
int tree_diameter(const vector<vector<int>>&g){auto a=farthest(0,g).first;return farthest(a,g).second;}
```

TYPE 9. BST search/insert/delete

Problem: Search, insert, and delete values while preserving BST ordering.

```cpp
// Replace a deleted node with its inorder successor when both children exist.
TreeNode* bst_insert(TreeNode* r,int x){if(!r)return new
 ↪ TreeNode(x);if(x<r->val)r->left=bst_insert(r->left,x);else
 ↪ if(x>r->val)r->right=bst_insert(r->right,x);return r;}
TreeNode* bst_search(TreeNode*r,int x){while(r&&r->val!=x)r=x<r->val?r->left:r->right;return r;}
TreeNode* bst_delete(TreeNode*r,int x){
    if(!r)return r;if(x<r->val)r->left=bst_delete(r->left,x);else
     ↪ if(x>r->val)r->right=bst_delete(r->right,x);else{
        if(!r->left){auto q=r->right;delete r;return q;}if(!r->right){auto q=r->left;delete r;return q;}
        auto q=r->right;while(q->left)q=q->left;r->val=q->val;r->right=bst_delete(r->right,q->val);
    }return r;
}
```

TYPE 10. Hard transfer - 2612 - Minimum Reverse Operations

Problem: From one starting index, find the minimum reversals needed to reach every position while respecting banned positions. **Link:** LeetCode problem

```cpp
// Reachable targets from i form one parity interval; erase each discovered index immediately so total work
 ↪ stays near O(n log n).
class Solution {
public:
    vector<int> minReverseOperations(int n, int p, vector<int>& banned, int k) {
        vector<int> ans(n, -1), ban(n, 0); set<int> unseen[2];
        for (int x : banned) ban[x] = 1;
        for (int i = 0; i < n; ++i) if (!ban[i] && i != p) unseen[i & 1].insert(i);
        queue<int> q; q.push(p); ans[p] = 0;
        while (!q.empty()) {
            int i = q.front(); q.pop();
            int lo = max(i - k + 1, k - i - 1);
            int hi = min(i + k - 1, 2 * n - k - i - 1);
            auto& s = unseen[lo & 1];
            auto it = s.lower_bound(lo);
            while (it != s.end() && *it <= hi) {
                int j = *it; it = s.erase(it); ans[j] = ans[i] + 1; q.push(j);
            }
        }
        return ans;
    }
};
```

SOLVE 11. Hard transfer - 3609 - Minimum Moves to Reach Target in Grid

Problem: Find the minimum moves to reach a target grid point under the problem's coordinate transformation rule. **Link:** LeetCode problem

READ

0-1 BFS

```cpp
// For weights 0/1, push zero-weight relaxations to the front and one-weight relaxations to the back.
vector<int> bfs01(int s,const vector<vector<pair<int,int>>>& g){
    int n=g.size();vector<int>d(n,1e9);deque<int>q;q.push_back(s);d[s]=0;
    while(!q.empty()){int u=q.front();q.pop_front();for(auto
     ↪  [v,w]:g[u])if(d[v]>d[u]+w){d[v]=d[u]+w;if(w)q.push_back(v);else q.push_front(v);}}
    return d;
}
```

Reading-only fallbacks

Use these only when the constraints force them; they are not part of the normal typing rotation.

READ **Bellman-Ford**

```
// Guard INF before relaxation; an update on the nth pass means a reachable negative cycle exists.
pair<bool,vector<long long>> bellman_ford(int s,int n,const vector<array<long long,3>>& e){
    const long long INF=4'000'000'000'000'000'000LL;vector<long long>d(n,INF);d[s]=0;bool neg=false;
    for(int it=0;it<n;++it){bool any=false;for(auto
     ↪  [u,v,w]:e)if(d[u]<INF&&d[v]>d[u]+w)d[v]=d[u]+w,any=true;if(!any)break;if(it==n-1)neg=true;}
    return {neg,d};
}
```

READ **Floyd-Warshall**

```
// Keep k outermost so every update uses only paths whose intermediate vertices have already been processed.
void floyd(vector<vector<long long>>& d){
    int n=d.size();const long long INF=4'000'000'000'000'000'000LL;
    for(int k=0;k<n;++k)for(int i=0;i<n;++i)if(d[i][k]<INF)
        for(int j=0;j<n;++j)if(d[k][j]<INF)d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
}
```

Friday

Focus: DP / Backtracking

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min cards; 38-53 min Hard transfer; 53-60 min retype misses + one edge case. Alternate the DP set with backtracking on the final card block.

Read / recall first

- **State before code:** write exactly what `dp[i]` / `dp[i][j]` means.
- **0/1 vs unbounded:** the loop direction decides whether an item can be reused.
- **LIS:** tails array stores the smallest possible tail for each length.
- **Backtracking:** make -> recurse -> undo; prune only with a condition that is provably safe.

Hard rotation: **3957 Maximum Sum of M Non-Overlapping Subarrays II** - *Stress.* Prefix DP plus monotonic/deque and penalty optimization ideas; read, do not memorize.; **3276 Select Cells in Grid With Maximum Score** - *Stress.* Small-state bitmask DP; check constraints before rejecting 2^k states.; **3509 Maximum Product of Subsequences With an Alternating Sum Equal to K** - *Stress.* DP state couples alternating-sum parity with product choice.; **3630 Partition Array for Maximum XOR and AND** - *Stress.* Small-n subset/bitmask DP; exploit the tiny combinatorial state space.; **3743 Maximize Cyclic Partition Score** - *Stress.* Reduce the cyclic partition to DP over segment/extrema choices.; **3704 Count No-Zero Pairs That Sum to N** - *Stress.* Digit DP with carry plus validity/alive flags.; **3256 Maximum Value Sum by Placing Three Rooks I** - *Solved benchmark.* Only a few top row/column candidates matter; prune instead of full enumeration.

Canonical cards - one copy: read once, then type/solve as marked

TYPE 1. 0/1 Knapsack

Problem: Choose each item at most once to maximize value under a capacity limit.

```
// Reverse capacity iteration prevents using the same item multiple times in one transition.
int knapsack01(const vector<int>& w,const vector<int>& val,int W){
    vector<int>dp(W+1);for(int i=0;i<(int)w.size();++i)for(int
 ↪  c=W;c>=w[i];--c)dp[c]=max(dp[c],dp[c-w[i]]+val[i]);return dp[W];
}
```

TYPE 2. Subset sum / partition

Problem: Determine whether array elements can form a required sum or equal partition.

```
// For 0/1 subset choice, update sums downward so one item is not counted twice.
bool subset_sum(const vector<int>& a,int S){
    vector<char>dp(S+1);dp[0]=1;for(int x:a)for(int s=S;s>=x;--s)dp[s]|=dp[s-x];return dp[S];
}
```

TYPE 3. Longest Increasing Subsequence

Problem: Find the longest strictly increasing subsequence.

```
// tails[len-1] is the smallest possible tail value of a subsequence of length len.
int lis(const vector<int>& a){
    vector<int>t;for(int x:a){auto it=lower_bound(t.begin(),t.end(),x);if(it==t.end())t.push_back(x);else
 ↪  *it=x;}return t.size();
}
```

TYPE 4. House Robber

Problem: Maximize value when adjacent houses cannot both be chosen.

```
// prev2 and prev track the best answer through the previous two positions.
long long house_robber(const vector<long long>&a){
    long long prev2=0,prev1=0;for(long long x:a){long long
     ↪  cur=max(prev1,prev2+x);prev2=prev1;prev1=cur;}return prev1;
}
```

TYPE 5. Longest Common Subsequence

Problem: Find the longest subsequence shared by two strings.

```cpp
// dp[j] is the best LCS length for the processed prefix of the first string and the first j characters of the
 ↪  second.
int lcs(const string&a,const string&b){
    vector<int>dp(b.size()+1);
    for(char x:a){int diag=0;for(int j=1;j<=(int)b.size();++j){int old=dp[j];if(x==b[j-1])dp[j]=diag+1;else
     ↪  dp[j]=max(dp[j],dp[j-1]);diag=old;}}
    return dp.back();
}
```

TYPE 6. Edit distance

Problem: Find the minimum insert/delete/replace operations to convert one string to another.

```cpp
// prev/cur rows are enough because each transition uses only left, up, and diagonal cells.
int edit_distance(const string&a,const string&b){
    vector<int>dp(b.size()+1);iota(dp.begin(),dp.end(),0);
    for(int i=1;i<=(int)a.size();++i){int diag=dp[0];dp[0]=i;for(int j=1;j<=(int)b.size();++j){int
     ↪  up=dp[j];dp[j]=a[i-1]==b[j-1]?diag+1:1+min({diag,dp[j-1],dp[j]});diag=up;}}
    return dp.back();
}
```

TYPE 7. Word Break

Problem: Determine whether a string can be segmented into dictionary words.

```cpp
// dp[i] means s[0..i) can be segmented; check every earlier cut that is reachable.
bool word_break(const string&s,const unordered_set<string>&dict){
    int n=s.size();vector<char>dp(n+1);dp[0]=1;for(int i=1;i<=n;++i)for(int
     ↪  j=0;j<i;++j)if(dp[j]&&dict.count(s.substr(j,i-j)))dp[i]=1;return dp[n];
}
```

TYPE 8. Decode Ways

Problem: Count the ways to decode a digit string into letters.

```cpp
// Zero can only appear as part of a valid 10..26 pair; it cannot start a one-digit decoding.
int decode_ways(const string&s){
    int n=s.size();if(!n||s[0]=='0')return 0;long long p2=1,p1=1;
    for(int i=1;i<n;++i){long long cur=0;if(s[i]!='0')cur+=p1;int
     ↪  x=10*(s[i-1]-'0')+s[i]-'0';if(x>=10&&x<=26)cur+=p2;p2=p1;p1=cur;}return p1;
}
```

TYPE 9. Unique paths / minimum path sum

```cpp
// For unique paths, each cell is top + left; for min path sum, replace addition with min transition.
long long unique_paths(int m,int n){
    vector<long long>dp(n);dp[0]=1;for(int i=0;i<m;++i)for(int j=1;j<n;++j)dp[j]+=dp[j-1];return dp[n-1];
}
long long min_path_sum(vector<vector<int>>g){
    int m=g.size(),n=g[0].size();for(int i=0;i<m;++i)for(int j=0;j<n;++j){if(i||j){int
     ↪  up=i?g[i-1][j]:INT_MAX/2,left=j?g[i][j-1]:INT_MAX/2;g[i][j]+=min(up,left);}}return g[m-1][n-1];
}
```

TYPE 10. N-Queens

Problem: Place n queens on an n x n board so that no two attack each other.

```
// col, diag1, and diag2 masks encode exactly the unavailable positions for the next row.
long long nqueens(int n){
    long long full=(1LL<<n)-1;
    function<long long(long long,long long,long long)>dfs=[&](long long c,long long d1,long long d2)->long
 ↪  long{
        if(c==full)return 1;long long ans=0,avail=full&~(c|d1|d2);
        while(avail){long long b=avail&-avail;avail-=b;ans+=dfs(c|b,(d1|b)<<1,(d2|b)>>1);}return ans;
    };
    return dfs(0,0,0);
}
```

TYPE 11. Hard transfer - 2945 - Find Maximum Non-decreasing Array Length

Problem: Split the array into the maximum number of contiguous parts whose sums are non-decreasing. **Link:** LeetCode problem

```
// The state is the number of parts made so far plus the start of the last merged block; prefix sums make the
 ↪  next feasible cut a lower_bound.
class Solution {
public:
    int findMaximumLength(vector<int>& nums) {
        int n = nums.size(); vector<long long> pref(n + 1), bestLeft(n + 2), dp(n + 1);
        for (int i = 0; i < n; ++i) pref[i + 1] = pref[i] + nums[i];
        for (int i = 1; i <= n; ++i) {
            bestLeft[i] = max(bestLeft[i], bestLeft[i - 1]);
            long long need = 2 * pref[i] - pref[bestLeft[i]];
            int r = lower_bound(pref.begin(), pref.end(), need) - pref.begin();
            dp[i] = dp[bestLeft[i]] + 1;
            bestLeft[r] = i;
        }
        return dp[n];
    }
};
```

Saturday

Focus: Fenwick / Segment Tree

Read: 10-20 min - use the recall cues first. **Type:** 8-38 min Segment Tree/Fenwick cards; 38-53 min Hard transfer; 53-60 min retype misses. Focus on reconstructing node state and lazy composition.

Read / recall first

- **Fenwick:** prefix aggregation with lowbit jumps; great for point updates + prefix/range queries.
- **Generic Segment Tree:** define the node summary, merge, identity, build/update/query.
- **Lazy:** define `apply(tag,node)` and tag composition before writing the recursion.
- **Special nodes:** max-subarray needs sum/prefix/suffix/best; min+frequency needs the winning minimum plus its count.

READ **Segment Tree operation map**

Aggregate	Identity	Associative	Commutative	Typical use
Sum	`0`	Yes	Yes	range sum
XOR	`0`	Yes	Yes	parity / xor range
GCD	`0`	Yes	Yes	divisibility / gcd queries
Min	`+INF`	Yes	Yes	range minimum
Max	`-INF`	Yes	Yes	range maximum
String concat	`""`	Yes	No	order-sensitive merge
Matrix multiply	`I`	Yes	No	state transitions

For non-commutative merges, query with separate left/right accumulators so order is preserved. For lazy propagation, define node update + tag composition first; composition order is chronological, not alphabetical.

Hard rotation: **3569 Maximize Count of Distinct Primes After Split** - *Stress.* SPF + occurrence positions + lazy Segment Tree; a specialized range-update pattern.; **4055 Count Shadow Pairs II** - *Stress.* Crossing-count divide and conquer with coordinate compression and Fenwick.; **3260 Find the Largest Palindrome Divisible by K** - *Solved benchmark.* Constructive digit DP over remainder; the left half determines the mirrored half.; **4007 Widest Possible Fence** - *Solved benchmark.* Frequency map + complementary pairing; enumerate candidate heights instead of building planks.

Canonical cards - one copy: read once, then type/solve as marked

TYPE **1. Fenwick Tree**

```
struct BIT {
    int n; vector<long long> ft;
    BIT(int n) : n(n), ft(n + 1, 0) {}
    void upd(int x, long long v) { for (; x <= n; x += x & -x) ft[x] += v; }
    long long sum(int x) { long long res = 0; for (; x; x -= x & -x) res += ft[x]; return res; }
    long long qry(int l, int r) { return sum(r) - sum(l - 1); }
};
```

TYPE **2. Generic associative Segment Tree**

```
// Only associativity is required; commutativity is optional, and the identity is used for disjoint query
 ↪ parts.
template<class T,class F>
struct SegTree {
    int n; T id; F op; vector<T> st;
    SegTree(const vector<T>& a,T id,F op):n(a.size()),id(id),op(op),st(2*n,id){
        for(int i=0;i<n;++i)st[n+i]=a[i];
        for(int i=n-1;i;--i)st[i]=op(st[i<<1],st[i<<1|1]);
    }
    void setPoint(int p,T x){
        st[p+=n]=x;
```

```
        for(p>>=1;p;p>>=1)st[p]=op(st[p<<1],st[p<<1|1]);
    }
    T query(int l,int r){
        T L=id,R=id;
        for(l+=n,r+=n;l<r;l>>=1,r>>=1){
            if(l&1)L=op(L,st[l++]);
            if(r&1)R=op(st[--r],R);
        }
        return op(L,R);
    }
};
```

TYPE 3. Generic Lazy Segment Tree

```
// Associative merge is required; lazy tags must form a composable action on nodes, with order preserved for
 ↪ noncommutative updates.
template<class Node,class Tag,class Merge,class Apply,class Compose>
struct LazySegTree {
    int n; Node idNode; Tag idTag; Merge merge; Apply apply; Compose compose;
    vector<Node> st; vector<Tag> lazy; vector<char> has;
    LazySegTree(const vector<Node>&a,Node idNode,Tag idTag,Merge merge,Apply apply,Compose
 ↪ compose):n(a.size()),idNode(idNode),idTag(idTag),merge(merge),apply(apply),compose(compose),st(4*n,idNode
 ↪ ),lazy(4*n,idTag),has(4*n){
        build(1,0,n-1,a);
    }
    void build(int v,int l,int r,const vector<Node>&a){
        if(l==r){st[v]=a[l];return;}
        int m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=merge(st[v*2],st[v*2+1]);
    }
    void applyTag(int v,const Tag&t,int len){apply(st[v],t,len);if(has[v])lazy[v]=compose(t,lazy[v]);else
     ↪ lazy[v]=t,has[v]=1;}
    void push(int v,int l,int r){if(!has[v]||l==r)return;int
     ↪ m=(l+r)/2;applyTag(v*2,lazy[v],m-l+1);applyTag(v*2+1,lazy[v],r-m);lazy[v]=idTag;has[v]=0;}
    void update(int ql,int qr,const Tag&t,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){applyTag(v,t,r-l+1);return;}
        push(v,l,r);int
 ↪ m=(l+r)/2;update(ql,qr,t,v*2,l,m);update(ql,qr,t,v*2+1,m+1,r);st[v]=merge(st[v*2],st[v*2+1]);
    }
    Node query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return idNode;if(ql<=l&&r<=qr)return st[v];
        push(v,l,r);int m=(l+r)/2;return merge(query(ql,qr,v*2,l,m),query(ql,qr,v*2+1,m+1,r));
    }
};
```

TYPE 4. Lazy Segment Tree: range add + range sum

```
// For a node covering len elements, adding x increases its sum by x*len and its lazy tag by x.
struct LazySum {
    int n; vector<long long> st,lz;
    LazySum(int n):n(n),st(4*n),lz(4*n){}
    void apply(int v,int l,int r,long long x){st[v]+=x*(r-l+1);lz[v]+=x;}
    void push(int v,int l,int r){if(!lz[v]||l==r)return;int
     ↪ m=(l+r)/2;apply(v*2,l,m,lz[v]);apply(v*2+1,m+1,r,lz[v]);lz[v]=0;}
    void add(int ql,int qr,long long x,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;
        if(ql<=l&&r<=qr){apply(v,l,r,x);return;}
        push(v,l,r);int m=(l+r)/2;add(ql,qr,x,v*2,l,m);add(ql,qr,x,v*2+1,m+1,r);st[v]=st[v*2]+st[v*2+1];
    }
    long long query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;
        if(ql<=l&&r<=qr)return st[v];
        push(v,l,r);int m=(l+r)/2;return query(ql,qr,v*2,l,m)+query(ql,qr,v*2+1,m+1,r);
    }
```

```
};
```

TYPE 5. Lazy Segment Tree: range add + min/max/sum

```
// A range add shifts sum, minimum, and maximum by predictable amounts, so one lazy tag is sufficient.
struct AddMinMax {
    struct Node{long long sum,mn,mx;int len;};
    int n; vector<Node> st; vector<long long> lz;
    AddMinMax(const vector<long long>& a):n(a.size()),st(4*n),lz(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>& a){
        if(l==r){st[v]={a[l],a[l],a[l],1};return;}int m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);pull(v);
    }
    void pull(int v){st[v].sum=st[v*2].sum+st[v*2+1].sum;st[v].mn=min(st[v*2].mn,st[v*2+1].mn);st[v].mx=max(
     ↪  st[v*2].mx,st[v*2+1].mx);st[v].len=st[v*2].len+st[v*2+1].len;}
    void apply(int v,long long x){st[v].sum+=x*st[v].len;st[v].mn+=x;st[v].mx+=x;lz[v]+=x;}
    void push(int v){if(lz[v]){apply(v*2,lz[v]);apply(v*2+1,lz[v]);lz[v]=0;}}
    void add(int ql,int qr,long long x,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){apply(v,x);return;}push(v);int
         ↪  m=(l+r)/2;add(ql,qr,x,v*2,l,m);add(ql,qr,x,v*2+1,m+1,r);pull(v);
    }
    Node query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return {0,LLONG_MAX,LLONG_MIN,0};
        if(ql<=l&&r<=qr)return st[v];push(v);int m=(l+r)/2;auto
         ↪  a=query(ql,qr,v*2,l,m),b=query(ql,qr,v*2+1,m+1,r);if(!a.len)return b;if(!b.len)return a;return
         ↪  {a.sum+b.sum,min(a.mn,b.mn),max(a.mx,b.mx),a.len+b.len};
    }
};
```

TYPE 6. Segment Tree: maximum subarray sum

```
// Four aggregate fields are exactly the information needed to combine two adjacent segments.
struct MaxSubSeg {
    struct N{long long sum,pref,suff,best;};
    int n; vector<N> st;
    N mergeN(N a,N b){return
 ↪  {a.sum+b.sum,max(a.pref,a.sum+b.pref),max(b.suff,b.sum+a.suff),max({a.best,b.best,a.suff+b.pref})};}
    MaxSubSeg(const vector<long long>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>& a){if(l==r){st[v]={a[l],a[l],a[l],a[l]};return;}int
     ↪  m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=mergeN(st[v*2],st[v*2+1]);}
    void setPoint(int p,long long x,int v=1,int l=0,int
     ↪  r=-1){if(r<0)r=n-1;if(l==r){st[v]={x,x,x,x};return;}int m=(l+r)/2;if(p<=m)setPoint(p,x,v*2,l,m);else
     ↪  setPoint(p,x,v*2+1,m+1,r);st[v]=mergeN(st[v*2],st[v*2+1]);}
};
```

TYPE 7. Segment Tree: minimum value + frequency

```
// On equal minima, add frequencies; on unequal minima, keep only the smaller side.
struct MinCountSeg {
    using N=pair<int,int>;
    int n; vector<N> st;
    static N op(N a,N b){if(a.first<b.first)return a;if(b.first<a.first)return b;return
     ↪  {a.first,a.second+b.second};}
    MinCountSeg(const vector<int>& a):n(a.size()),st(2*n){for(int i=0;i<n;++i)st[n+i]={a[i],1};for(int
 ↪  i=n-1;i;--i)st[i]=op(st[i<<1],st[i<<1|1]);}
    N query(int l,int r){N L={INT_MAX,0},R={INT_MAX,0};for(l+=n,r+=n;l<r;l>>=1,r>>=1){if(l&1)L=op(L,st[l++]);
 ↪  if(r&1)R=op(st[--r],R);}return op(L,R);}
};
```

TYPE 8. Hard transfer - 3901 - GCD Segment Tree variant (not full problem solution)

Problem: Practice the reusable GCD Segment Tree needed inside 3901; this card is **not** the complete 3901 query solution.
Link: LeetCode problem

```
// GCD is associative with identity 0, so it fits the generic Segment Tree directly; the full 3901 solution additionally tracks divisibility/prime-factor coverage.
struct GcdSeg {
    int n; vector<long long> st;
    GcdSeg(const vector<long long>& a) : n(a.size()), st(4 * n + 5) { build(1, 0, n - 1, a); }
    void build(int v, int l, int r, const vector<long long>& a) { if (l == r) { st[v] = a[l]; return; } int m=(l+r)/2; build(v*2,l,m,a); build(v*2+1,m+1,r,a); st[v]=gcd(st[v*2],st[v*2+1]); }
    void update(int v, int l, int r, int p, long long x) { if (l == r) { st[v] = x; return; } int m=(l+r)/2; if(p<=m) update(v*2,l,m,p,x); else update(v*2+1,m+1,r,p,x); st[v]=gcd(st[v*2],st[v*2+1]); }
    void update(int p, long long x) { update(1,0,n-1,p,x); }
    long long query(int v, int l, int r, int ql, int qr) { if(qr<l || r<ql) return 0; if(ql<=l && r<=qr) return st[v]; int m=(l+r)/2; return gcd(query(v*2,l,m,ql,qr), query(v*2+1,m+1,r,ql,qr)); }
    long long query(int l, int r) { return l > r ? 0 : query(1,0,n-1,l,r); }
};
```

TYPE 9. Hard transfer - 3920 - Maximize Fixed Points After Deletions

Problem: Delete elements to maximize how many remaining positions become fixed points. **Link:** LeetCode problem

```
// Group equal values first so states from the same value do not chain; d=i-value is the monotone coordinate used by the Segment Tree.
struct FixedPointSeg {
    int l, r, ma; FixedPointSeg *left, *right;
    FixedPointSeg(int le, int ri) : l(le), r(ri), ma(0), left(nullptr), right(nullptr) {
        if (l != r) { int m = l + (r - l) / 2; left = new FixedPointSeg(l, m); right = new FixedPointSeg(m + 1, r); }
    }
    void update(int n, int x) {
        if (l <= n && n <= r) { ma = max(ma, x); if (left) left->update(n, x); if (right) right->update(n, x); }
    }
    int query(int ql, int qr) {
        if (ql <= l && r <= qr) return ma;
        if (l > qr || r < ql) return 0;
        return max(left->query(ql, qr), right->query(ql, qr));
    }
};

class Solution {
public:
    int maxFixedPoints(vector<int>& nums) {
        int mx = *max_element(nums.begin(), nums.end());
        vector<vector<int>> groups(mx + 1);
        for (int i = 0; i < (int)nums.size(); ++i) if (nums[i] <= i) groups[nums[i]].push_back(i - nums[i]);
        FixedPointSeg seg(0, (int)nums.size());
        int ans = 0;
        for (int v = 0; v <= mx; ++v) {
            vector<pair<int,int>> batch;
            for (int d : groups[v]) { int cur = seg.query(0, d) + 1; batch.push_back({d, cur}); ans = max(ans, cur); }
            for (auto [d, cur] : batch) seg.update(d, cur);
        }
        return ans;
    }
};
```

Sunday

Focus: C++ / Quant / Hard Review / Advanced

Read: 10-30 min - use the recall cues first. **Type:** 8-28 min C++ systems lab; 28-48 min timed benchmark; 48-55 min STL/quant drill; 55-60 min failure retype. Every 3-4 Sundays, swap in the optional low-level drill.

Read / recall first

- **C++ systems:** ownership, lifetime, `const`, virtual dispatch, templates, move semantics, and STL ordering.
- **Quant:** think in terms of order book state, stable ordering, pointer validity, and low-allocation data flow.
- **Hards:** use this day to reconstruct; do not add another long implementation unless the pattern is reusable.

Hard rotation: Advanced hard review: rotate 2-4 Hards - *Reading rule.* Do not finish all of them. Reconstruct invariant/state, why the obvious approach fails, the key transition, and complexity.

Canonical cards - one copy: read once, then type/solve as marked

TYPE **1. C++ Pointer + OOP Systems Lab**

```cpp
#include <bits/stdc++.h>
using namespace std;

class Instrument {
protected:
    long long id, px;
public:
    Instrument(long long id, long long px) : id(id), px(px) {}
    virtual ~Instrument() = default;
    long long getId() const { return id; }
    virtual long long value(long long qty) const = 0;
    virtual string kind() const = 0;
};

class FutureOrder : public Instrument {
public:
    FutureOrder(long long id, long long px) : Instrument(id, px) {}
    long long value(long long qty) const override { return px * qty; }
    string kind() const override { return "FUT"; }
};

class OptionOrder : public Instrument {
    long long strike;
public:
    OptionOrder(long long id, long long px, long long strike) : Instrument(id, px), strike(strike) {}
    long long value(long long qty) const override { return max(0LL, px - strike) * qty; }
    string kind() const override { return "OPT"; }
};

struct TreeNode {
    long long key;
    TreeNode* parent = nullptr;
    unique_ptr<TreeNode> left, right;
    explicit TreeNode(long long key) : key(key) {}
};

class SearchTree {
    unique_ptr<TreeNode> root;

    TreeNode* insert(unique_ptr<TreeNode>& node, TreeNode* parent, long long x) {
        if (!node) { node = make_unique<TreeNode>(x); node->parent = parent; return node.get(); }
        if (x < node->key) return insert(node->left, node.get(), x);
        if (x > node->key) return insert(node->right, node.get(), x);
        return node.get();
    }

    const TreeNode* find(const unique_ptr<TreeNode>& node, long long x) const {
```

```
        if (!node || node->key == x) return node.get();
        return x < node->key ? find(node->left, x) : find(node->right, x);
    }

    void inorder(const unique_ptr<TreeNode>& node, vector<long long>& out) const {
        if (!node) return;
        inorder(node->left, out); out.push_back(node->key); inorder(node->right, out);
    }

public:
    TreeNode* insert(long long x) { return insert(root, nullptr, x); }
    const TreeNode* find(long long x) const { return find(root, x); }
    vector<long long> sorted() const { vector<long long> out; inorder(root, out); return out; }
};

template<class T, class Compare = greater<T>>
class TopK {
    priority_queue<T, vector<T>, Compare> pq;
    size_t k;
public:
    explicit TopK(size_t k) : k(k) {}
    void add(const T& x) {
        if (k == 0) return;
        pq.push(x);
        if (pq.size() > k) pq.pop();
    }
    const T& worst() const { return pq.top(); }
};

class Portfolio {
    vector<unique_ptr<Instrument>> instruments;
    unordered_map<long long, Instrument*> byId;
public:
    void add(unique_ptr<Instrument> p) {
        long long id = p->getId();
        Instrument* raw = p.get();
        byId[id] = raw;
        instruments.push_back(move(p));
    }
    long long exposure(long long qty) const {
        long long ans = 0;
        for (const auto& p : instruments) ans += p->value(qty);
        return ans;
    }
    Instrument* get(long long id) const {
        auto it = byId.find(id);
        return it == byId.end() ? nullptr : it->second;
    }
};

int main() {
    Portfolio p;
    p.add(make_unique<FutureOrder>(1, 100));
    p.add(make_unique<OptionOrder>(2, 125, 110));

    SearchTree t;
    t.insert(8); t.insert(3); t.insert(10); t.insert(1); t.insert(6);
    auto order = t.sorted();

    cout << p.exposure(10) << ' ' << order.size() << '\n';
    return 0;
}
```

TYPE 2. Quant-style order-book / STL drill

```cpp
struct Quote {
    long long id, px, qty;
};

class MiniOrderBook {
    map<long long, deque<Quote>, greater<long long>> bids;
    map<long long, deque<Quote>> asks;

public:
    void addBid(Quote q) { bids[q.px].push_back(q); }
    void addAsk(Quote q) { asks[q.px].push_back(q); }
    const Quote* bestBid() const {
        if (bids.empty() || bids.begin()->second.empty()) return nullptr;
        return &bids.begin()->second.front();
    }
    const Quote* bestAsk() const {
        if (asks.empty() || asks.begin()->second.empty()) return nullptr;
        return &asks.begin()->second.front();
    }
    void cancelBid(long long px) {
        auto it = bids.find(px);
        if (it != bids.end() && !it->second.empty()) it->second.pop_front();
        if (it != bids.end() && it->second.empty()) bids.erase(it);
    }
    void cancelAsk(long long px) {
        auto it = asks.find(px);
        if (it != asks.end() && !it->second.empty()) it->second.pop_front();
        if (it != asks.end() && it->second.empty()) asks.erase(it);
    }
};
```

READ 3. Optional low-level drill - every 3-4 Sundays

```cpp
template<class T, size_t N>
class ObjectPool {
    alignas(T) unsigned char storage[N][sizeof(T)];
    bool used[N]{};
public:
    template<class... Args> T* create(Args&&... args) {
        for (size_t i = 0; i < N; ++i) if (!used[i]) {
            used[i] = true;
            return new (storage[i]) T(forward<Args>(args)...);
        }
        return nullptr;
    }
    void destroy(T* p) {
        if (!p) return;
        auto base = reinterpret_cast<uintptr_t>(&storage[0][0]);
        auto cur = reinterpret_cast<uintptr_t>(p);
        auto span = sizeof(storage);
        if (cur < base || cur >= base + span || (cur - base) % sizeof(T)) return;
        size_t i = (cur - base) / sizeof(T);
        if (!used[i]) return;
        p->~T(); used[i] = false;
    }
};
```

Benchmark rule

Every 3-4 weeks, randomly choose one problem marked **Solved benchmark** anywhere in the week and give yourself the original 18-27 minute limit. Stop after the accepted solution and record the one transition that was hardest to recall.

Quant / developer focus: object lifetime, ownership, pointers/references, `const`, `virtual/override`, templates, move

semantics, custom ordering, STL, tree nodes, RAII.

Optional break-glass reading

The advanced vault below is **read-only unless the OA signals it**. Know the trigger and the core invariant; do not spend normal meal time memorizing these implementations.

Advanced Code Vault - break-glass implementations

READ Advanced Vault is reference-only. Learn the trigger, invariant, and complexity; do not memorize it in the normal typing cycle.

These implementations were deliberately removed from Blind Typing. Use this section only when a placement/OA, target role, or a failed problem makes the technique relevant. The main meal-reading path remains the earlier compact sections.

READ 1. Mo's Algorithm (Sqrt Decomposition)

Static range-query technique; remember the moving-window state machine before the ordering heuristic.

```
const int SQRT = 500;
struct MoQuery {
    int l, r, id;
    bool operator<(const MoQuery& o) const {
        return l / SQRT == o.l / SQRT ? ((l / SQRT) & 1 ? r < o.r : r > o.r) : l < o.l;
    }
};
```

READ 2. Manacher's Algorithm

Linear odd/even palindrome-radius template; the returned array is the radius at each transformed-string center.

```
vector<int> manacher(string s) {
    string t = "^";
    for (char c : s) t += string("#") + c;
    t += "#$";
    vector<int> p(t.size());
    int center = 0, right = 0;
    for (int i = 1; i + 1 < (int)t.size(); ++i) {
        int mir = 2 * center - i;
        if (i < right) p[i] = min(right - i, p[mir]);
        while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) ++p[i];
        if (i + p[i] > right) center = i, right = i + p[i];
    }
    return p;
}
```

READ 3. Lyndon Factorization (Min Cyclic Shift)

Use for minimum cyclic rotation / Lyndon decomposition; specialized but compact once understood.

```
string min_cyclic_string(string s) {
    s += s; int n = s.size(), i = 0, ans = 0;
    while (i < n / 2) {
        ans = i; int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) { s[k] < s[j] ? k = i : k++; j++; }
        while (i <= k) i += j - k;
    }
    return s.substr(ans, n / 2);
}
```

READ 4. Combinatorics (nCr, Catalan, Stars & Bars) & Mint Struct

Reusable modular-combinatorics utilities; read the assumptions on MOD before using inverse by Fermat.

```
const int MOD = 1e9 + 7;
struct Mint {
    int v;
    Mint(long long x = 0) : v((x % MOD + MOD) % MOD) {}
    Mint& operator+=(Mint o) { if ((v += o.v) >= MOD) v -= MOD; return *this; }
    Mint& operator-=(Mint o) { if ((v -= o.v) < 0) v += MOD; return *this; }
    Mint& operator*=(Mint o) { v = (long long)v * o.v % MOD; return *this; }
    friend Mint operator+(Mint a, Mint b) { return a += b; }
    friend Mint operator-(Mint a, Mint b) { return a -= b; }
    friend Mint operator*(Mint a, Mint b) { return a *= b; }
    Mint pow(long long k) const {
        Mint res = 1, a = *this;
        while (k) { if (k & 1) res *= a; a *= a; k >>= 1; }
        return res;
    }
    Mint inv() const { return pow(MOD - 2); }
    Mint& operator/=(Mint o) { return *this *= o.inv(); }
    friend Mint operator/(Mint a, Mint b) { return a /= b; }
};

vector<Mint> fact, invf;
void precompute(int n) {
    fact.assign(n + 1, 1); invf.assign(n + 1, 1);
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * Mint(i);
    invf[n] = fact[n].inv();
    for (int i = n - 1; i > 0; i--) invf[i] = invf[i + 1] * Mint(i + 1);
}

Mint nCk(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * invf[k] * invf[n - k];
}
Mint catalan(int n) { return nCk(2 * n, n) * Mint(n + 1).inv(); }
Mint stars_bars(int n, int k) { return nCk(n + k - 1, k - 1); }
```

READ 5. Matrix Exponentiation

Use for linear recurrences and fixed-length path counting; adapt the matrix size and multiplication operation.

```
template<size_t N, long long M = 1000000007LL>
struct Matrix {
    long long m[N][N] = {0};
    void identity() { for (size_t i = 0; i < N; ++i) m[i][i] = 1; }
    Matrix operator*(const Matrix& b) const {
        Matrix res;
        for (size_t i = 0; i < N; ++i) for (size_t k = 0; k < N; ++k) for (size_t j = 0; j < N; ++j)
            res.m[i][j] = (res.m[i][j] + m[i][k] * b.m[k][j]) % M;
        return res;
    }
    Matrix pow(long long p) const {
        Matrix res, base = *this; res.identity();
        while (p > 0) { if (p & 1) res = res * base; base = base * base; p >>= 1; }
        return res;
    }
};
```

READ 6. Tarjan's Bridges & Articulation Points

DFS low-link template; bridges use low[v] > tin[u], articulation uses low[v] >= tin[u].

```
int timer = 0;
vector<int> tin, low; vector<bool> is_art; vector<array<int,2>> bridges;

void dfs_bridges(int u, int p, const vector<vector<int>>& adj) {
    tin[u] = low[u] = timer++; int child = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (tin[v] != -1) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs_bridges(v, u, adj);
            low[u] = min(low[u], low[v]);
            child++;
            if (low[v] > tin[u]) bridges.push_back({u, v});
            if (low[v] >= tin[u] && p != -1) is_art[u] = 1;
        }
    }
    if (p == -1 && child > 1) is_art[u] = 1;
}
```

READ 7. Euler Tour Technique (Tree Flattening)

Turns subtree queries into interval queries; combine with Fenwick/Segment Tree.

```
int timer = 0;
vector<int> tin, tout;
void euler_tour(int u, int p, const vector<vector<int>>& adj) {
    tin[u] = ++timer;
    for (int v : adj[u]) {
        if (v != p) euler_tour(v, u, adj);
    }
    tout[u] = timer;
}
```

READ 8. Bitmask DP Scaffold (Assignments / TSP)

Use when n is small enough for 2^n states; the assignment index is usually popcount(mask).

```
void bitmask_dp(int n, vector<vector<int>>& cost) {
    vector<int> dp(1 << n, 1e9);
    dp[0] = 0;
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (dp[mask] == 1e9) continue;
        int bits_active = __builtin_popcount(mask);
        for (int i = 0; i < n; ++i) {
            if (!(mask & (1 << i))) {
                int next_mask = mask | (1 << i);
                dp[next_mask] = min(dp[next_mask], dp[mask] + cost[bits_active][i]);
            }
        }
    }
}
```

READ

9. Suffix Array

Doubling construction; read when suffix ordering is required rather than using hashing/KMP.

```
vector<int> suffix_array(string s) {
    s += "$"; int n = s.size(); vector<int> p(n), c(n); vector<pair<char,int>> a(n);
    for (int i = 0; i < n; i++) a[i] = {s[i], i}; sort(a.begin(), a.end());
    for (int i = 0; i < n; i++) p[i] = a[i].second;
    for (int i = 1; i < n; i++) c[p[i]] = a[i].first == a[i - 1].first ? c[p[i - 1]] : c[p[i - 1]] + 1;

    for (int k = 0; (1 << k) < n; k++) {
        for (int i = 0; i < n; i++) p[i] = (p[i] - (1 << k) + n) % n;
        vector<int> np(n), nc(n), cnt(n, 0), pos(n);
        for (int x : p) cnt[c[x]]++;
        for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
        for (int x : p) np[pos[c[x]]++] = x;
        p = np; nc[p[0]] = 0;
        for (int i = 1; i < n; i++) {
            pair<int,int> cur = {c[p[i]], c[(p[i] + (1 << k)) % n]};
            pair<int,int> pre = {c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]};
            nc[p[i]] = cur == pre ? nc[p[i - 1]] : nc[p[i - 1]] + 1;
        }
        c = nc;
    }
    return vector<int>(p.begin() + 1, p.end());
}
```

READ

10. Heavy-Light Decomposition (HLD)

Maps tree paths to O(log n) chain intervals; normally paired with a Segment Tree.

```
struct HLD {
    int n, timer; vector<int> sz, top, dep, par, in, out; vector<vector<int>> adj;
    HLD(int n) : n(n), timer(0), sz(n, 1), top(n), dep(n), par(n), in(n), out(n), adj(n) {}
    void dfs_sz(int u = 0, int p = -1, int d = 0) {
        par[u] = p; dep[u] = d; sz[u] = 1;
        if (adj[u].size() && adj[u][0] == p) swap(adj[u][0], adj[u].back());
        for (int& v : adj[u]) if (v != p) {
            dfs_sz(v, u, d + 1); sz[u] += sz[v];
            if (sz[v] > sz[adj[u][0]]) swap(v, adj[u][0]);
        }
    }
    void dfs_hld(int u = 0, int p = -1, int t = 0) {
        in[u] = timer++; top[u] = t;
        for (int v : adj[u]) if (v != p) dfs_hld(v, u, v == adj[u][0] ? t : v);
        out[u] = timer;
    }
    int lca(int u, int v) {
        while (top[u] != top[v]) {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = par[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }
};
```

READ 11. Meet in the Middle

Split n into halves and enumerate 2^(n/2) states; useful for n around 40-46.

```
long long meet_in_the_middle(const vector<long long>& arr, long long target) {
    int n = arr.size(), m1 = n / 2, m2 = n - m1;
    auto get_sums = [&](int st, int len) {
        vector<long long> sums;
        for (int i = 0; i < (1 << len); i++) {
            long long cur = 0;
            for (int j = 0; j < len; j++) if (i & (1 << j)) cur += arr[st + j];
            sums.push_back(cur);
        }
        sort(sums.begin(), sums.end()); return sums;
    };
    auto sum1 = get_sums(0, m1), sum2 = get_sums(m1, m2);
    long long ans = 0;
    for (long long s1 : sum1)
        ans += upper_bound(sum2.begin(), sum2.end(), target - s1) -
               lower_bound(sum2.begin(), sum2.end(), target - s1);
    return ans;
}
```

READ 12. BigInt (Rule of Five & Memory Leaks Test)

C++ lifetime drill: destructor, deep copy, move constructor, and move assignment.

```
class BigInt {
    int* data; size_t sz, cap;
public:
    BigInt(size_t c = 16) : sz(0), cap(c), data(new int[c]()) {}
    ~BigInt() { delete[] data; }

    BigInt(const BigInt& o) : sz(o.sz), cap(o.cap) {
        data = new int[cap]();
        for (size_t i = 0; i < sz; ++i) data[i] = o.data[i];
    }
    BigInt& operator=(const BigInt& o) {
        if (this != &o) {
            delete[] data;
            sz = o.sz; cap = o.cap;
            data = new int[cap]();
            for (size_t i = 0; i < sz; ++i) data[i] = o.data[i];
        }
        return *this;
    }

    BigInt(BigInt&& o) noexcept : data(o.data), sz(o.sz), cap(o.cap) {
        o.data = nullptr; o.sz = o.cap = 0;
    }
    BigInt& operator=(BigInt&& o) noexcept {
        if (this != &o) {
            delete[] data; data = o.data; sz = o.sz; cap = o.cap;
            o.data = nullptr; o.sz = o.cap = 0;
        }
        return *this;
    }
};
```

READ 13. CRTP & Cache Line Alignment (Low Latency)

Static polymorphism and alignment vocabulary for low-latency C++ interviews.

```cpp
// Static Polymorphism (Zero VTable overhead)
template <typename Derived>
struct MatchingEngine {
    void process() {
        static_cast<Derived*>(this)->execute();
    }
};

struct CryptoEngine : public MatchingEngine<CryptoEngine> {
    void execute() { /* Fast path */ }
};

struct alignas(64) OrderBookCounter {
    uint64_t fills;
};
```

READ 14. Lazy Segment Tree: range assign + range sum

Assignment overrides an existing lazy add/set state; read tag precedence carefully.

```cpp
// A pending assignment must be pushed before a pending add; assignment replaces the whole segment value.
struct AssignSum {
    int n; vector<long long> st,add; vector<long long> setv; vector<char> hasSet;
    AssignSum(int n):n(n),st(4*n),add(4*n),setv(4*n),hasSet(4*n){}
    void applySet(int v,int l,int r,long long x){st[v]=x*(r-l+1);setv[v]=x;hasSet[v]=1;add[v]=0;}
    void applyAdd(int v,int l,int r,long long x){st[v]+=x*(r-l+1);if(hasSet[v])setv[v]+=x;else add[v]+=x;}
    void push(int v,int l,int r){
        if(l==r){hasSet[v]=0;add[v]=0;return;}
        int m=(l+r)/2;
        if(hasSet[v]){applySet(v*2,l,m,setv[v]);applySet(v*2+1,m+1,r,setv[v]);hasSet[v]=0;}
        if(add[v]){applyAdd(v*2,l,m,add[v]);applyAdd(v*2+1,m+1,r,add[v]);add[v]=0;}
    }
    void update(int ql,int qr,long long x,bool isSet,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return;
        if(ql<=l&&r<=qr){if(isSet)applySet(v,l,r,x);else applyAdd(v,l,r,x);return;}
        push(v,l,r);int
 ↪  m=(l+r)/2;update(ql,qr,x,isSet,v*2,l,m);update(ql,qr,x,isSet,v*2+1,m+1,r);st[v]=st[v*2]+st[v*2+1];
    }
    long long query(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;if(ql<=l&&r<=qr)return st[v];push(v,l,r);int m=(l+r)/2;return
         ↪  query(ql,qr,v*2,l,m)+query(ql,qr,v*2+1,m+1,r);
    }
};
```

READ 15. Lazy Segment Tree: range affine transform + sum

Affine tags compose in chronological order; this is the canonical non-commutative lazy-tag example.

```
// For sum, (a*x+b) changes a segment sum to a*sum+b*len; compose new(a,b) after old(c,d) as (a*c,a*d+b).
struct AffineSum {
    struct Tag{long long a=1,b=0;}; struct Node{long long sum=0;int len=0;};
    int n;vector<Node>st;vector<Tag>lz;vector<char>has;
    AffineSum(const vector<long long>&a):n(a.size()),st(4*n),lz(4*n),has(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<long long>&a){if(l==r){st[v]={a[l],1};return;}int
     ↪  m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);pull(v);}
    void pull(int v){st[v].sum=st[v*2].sum+st[v*2+1].sum;st[v].len=st[v*2].len+st[v*2+1].len;}
    void apply(int v,Tag
     ↪  t){st[v].sum=t.a*st[v].sum+t.b*st[v].len;lz[v]={t.a*lz[v].a,t.a*lz[v].b+t.b};has[v]=1;}
    void push(int v){if(!has[v])return;apply(v*2,lz[v]);apply(v*2+1,lz[v]);lz[v]={1,0};has[v]=0;}
    void update(int ql,int qr,Tag t,int v=1,int l=0,int
     ↪  r=-1){if(r<0)r=n-1;if(qr<l||r<ql)return;if(ql<=l&&r<=qr){apply(v,t);return;}push(v);int
     ↪  m=(l+r)/2;update(ql,qr,t,v*2,l,m);update(ql,qr,t,v*2+1,m+1,r);pull(v);}
    long long query(int ql,int qr,int v=1,int l=0,int r=-1){if(r<0)r=n-1;if(qr<l||r<ql)return
     ↪  0;if(ql<=l&&r<=qr)return st[v].sum;push(v);int m=(l+r)/2;return
     ↪  query(ql,qr,v*2,l,m)+query(ql,qr,v*2+1,m+1,r);}
};
```

READ 16. Segment Tree Beats: range chmin + range sum/max

Ordinary lazy propagation cannot represent arbitrary chmin on a segment; Beats tracks max/second-max/count.

```
// Store max, second max, count of max, and sum; chmin only changes the current maximum layer.
struct SegBeats {
    struct N{long long mx,sx,sum;int cnt;};
    int n; vector<N> st;
    SegBeats(const vector<long long>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    N mergeN(N a,N b){
        N z;z.sum=a.sum+b.sum;
        if(a.mx==b.mx)z.mx=a.mx,z.cnt=a.cnt+b.cnt,z.sx=max(a.sx,b.sx);
        else if(a.mx>b.mx)z.mx=a.mx,z.cnt=a.cnt,z.sx=max(a.sx,b.mx);
        else z.mx=b.mx,z.cnt=b.cnt,z.sx=max(a.mx,b.sx);
        return z;
    }
    void build(int v,int l,int r,const vector<long long>& a){
        if(l==r){st[v]={a[l],LLONG_MIN,a[l],1};return;}int
         ↪  m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);st[v]=mergeN(st[v*2],st[v*2+1]);
    }
    void apply(int v,long long x){if(x>=st[v].mx)return;st[v].sum-=(st[v].mx-x)*st[v].cnt;st[v].mx=x;}
    void push(int v){apply(v*2,st[v].mx);apply(v*2+1,st[v].mx);}
    void chmin(int ql,int qr,long long x,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql||x>=st[v].mx)return;
        if(ql<=l&&r<=qr&&x>st[v].sx){apply(v,x);return;}
        if(l==r){apply(v,x);return;}
        push(v);int
 ↪  m=(l+r)/2;chmin(ql,qr,x,v*2,l,m);chmin(ql,qr,x,v*2+1,m+1,r);st[v]=mergeN(st[v*2],st[v*2+1]);
    }
    long long queryMax(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return LLONG_MIN;if(ql<=l&&r<=qr)return st[v].mx;push(v);int
         ↪  m=(l+r)/2;return max(queryMax(ql,qr,v*2,l,m),queryMax(ql,qr,v*2+1,m+1,r));
    }
    long long querySum(int ql,int qr,int v=1,int l=0,int r=-1){
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;if(ql<=l&&r<=qr)return st[v].sum;push(v);int m=(l+r)/2;return
         ↪  querySum(ql,qr,v*2,l,m)+querySum(ql,qr,v*2+1,m+1,r);
    }
};
```

READ 17. Merge Sort Tree: count <= x in a range

Static range order-statistics structure; each covered node contributes a binary search.

```
// Each node stores a sorted vector; a query touches O(log n) nodes and binary-searches each one.
struct MergeSortTree {
    int n; vector<vector<int>> st;
    MergeSortTree(const vector<int>& a):n(a.size()),st(4*n){build(1,0,n-1,a);}
    void build(int v,int l,int r,const vector<int>& a){
        if(l==r){st[v]={a[l]};return;}int m=(l+r)/2;build(v*2,l,m,a);build(v*2+1,m+1,r,a);merge(st[v*2].begin
        ↪ (),st[v*2].end(),st[v*2+1].begin(),st[v*2+1].end(),back_inserter(st[v]));
    }
    int leq(int ql,int qr,int x,int v=1,int l=0,int r=-1) const {
        if(r<0)r=n-1;if(qr<l||r<ql)return 0;
        if(ql<=l&&r<=qr)return upper_bound(st[v].begin(),st[v].end(),x)-st[v].begin();
        int m=(l+r)/2;return leq(ql,qr,x,v*2,l,m)+leq(ql,qr,x,v*2+1,m+1,r);
    }
};
```

READ 18. Persistent Segment Tree: k-th number in subarray

Prefix roots represent versions; subtract root L-1 from root R to recover a subarray frequency tree.

```
// Root R minus root L-1 counts how many values fall in each value interval of the queried subarray.
struct PST {
    struct Node{int l=0,r=0,c=0;};
    vector<Node> tr; vector<int> roots,vals; int U;
    PST(int n,int U):tr(1),roots(n+1),U(U){}
    int clone(int x){tr.push_back(tr[x]);return tr.size()-1;}
    int add(int old,int l,int r,int p){
        int v=clone(old);++tr[v].c;if(l!=r){int m=(l+r)/2;if(p<=m)tr[v].l=add(tr[v].l,l,m,p);else
        ↪ tr[v].r=add(tr[v].r,m+1,r,p);}return v;
    }
    void build(const vector<int>& a){

↪ vals=a;sort(vals.begin(),vals.end());vals.erase(unique(vals.begin(),vals.end()),vals.end());U=vals.size();
        roots.assign(a.size()+1,0);for(int i=0;i<(int)a.size();++i)roots[i+1]=add(roots[i],0,U-1,lower_bound(
↪ vals.begin(),vals.end(),a[i])-vals.begin());
    }
    int kth(int A,int B,int l,int r,int k){
        if(l==r)return vals[l];int m=(l+r)/2,c=tr[tr[B].l].c-tr[tr[A].l].c;
        if(k<=c)return kth(tr[A].l,tr[B].l,l,m,k);return kth(tr[A].r,tr[B].r,m+1,r,k-c);
    }
};
```

READ 19. Eulerian Path (Hierholzer's Algorithm)

Constructs an Euler trail by consuming edges and appending vertices on backtrack.

```
// Hierholzer's Algorithm (Directed Graph)
vector<int> find_eulerian_path(int n, vector<vector<int>>& adj) {
    vector<int> path, out(n), in(n);
    long long edges = 0;
    for (int u = 0; u < n; ++u) {
        out[u] = (int)adj[u].size(); edges += out[u];
        for (int v : adj[u]) ++in[v];
    }
    int start = 0, finish = 0, start_node = 0;
    for (int i = 0; i < n; ++i) {
        if (out[i] - in[i] == 1) { ++start; start_node = i; }
        else if (in[i] - out[i] == 1) ++finish;
        else if (in[i] != out[i]) return {};
    }
    if (!(start == 0 && finish == 0) && !(start == 1 && finish == 1)) return {};
    if (start == 0 && finish == 0) {
        for (int i = 0; i < n; ++i) if (out[i]) { start_node = i; break; }
```

```
    }
    stack<int> st; st.push(start_node);
    while (!st.empty()) {
        int u = st.top();
        if (out[u] == 0) { path.push_back(u); st.pop(); }
        else { int v = adj[u][--out[u]]; st.push(v); }
    }
    reverse(path.begin(), path.end());
    return (long long)path.size() == edges + 1 ? path : vector<int>{};
}
```

READ 20. Closest Pair of Points (Divide & Conquer)

Geometry fallback; the sweep implementation keeps only points within the current distance band.

```
struct pt { double x, y; };
double closest_pair(vector<pt>& p) {
    if (p.size() < 2) return 0.0;
    sort(p.begin(), p.end(), [](const pt& a, const pt& b) {
        return a.x == b.x ? a.y < b.y : a.x < b.x;
    });
    multiset<pair<double, double>> s;
    double d = 1e18; int l = 0;
    for (int i = 0; i < (int)p.size(); ++i) {
        while (l < i && p[i].x - p[l].x > d) {
            s.erase(s.find({p[l].y, p[l].x})); ++l;
        }
        auto it = s.lower_bound({p[i].y - d, -1e18});
        for (; it != s.end() && it->first - p[i].y <= d; ++it)
            d = min(d, hypot(p[i].x - it->second, p[i].y - it->first));
        s.insert({p[i].y, p[i].x});
    }
    return d;
}
```

READ 21. Sum Over Subsets (SOS) DP & Variants

Subset/superset zeta transforms over 2^k states; learn the loop structure, then map it to the required transform.

```
void sos_dp(int max_bits, vector<int>& a) {
    vector<int> dp(1 << max_bits, 0);
    for (int x : a) dp[x]++;

    for (int i = 0; i < max_bits; i++) {
        for (int mask = 0; mask < (1 << max_bits); mask++) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }



        }
    }
}
```

READ 22. Gaussian Elimination (Linear Systems & Expected Value)

General linear-system solver; inspect pivoting and floating-point tolerances before reuse.

```cpp
// Gaussian Elimination O(N^3)
// Returns: 1 (Unique solution), 0 (No solution), INF (Infinite solutions)
int gauss(vector<vector<double>>& a, vector<double>& ans) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        if (abs(a[sel][col]) < 1e-9) continue;
        swap(a[sel], a[row]); where[col] = row;
        for (int i = 0; i < n; ++i) if (i != row) {
            double c = a[i][col] / a[row][col];
            for (int j = col; j <= m; ++j) a[i][j] -= a[row][j] * c;
        }
        ++row;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i = 0; i < n; ++i) {
        double sum = 0; for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > 1e-9) return 0;
    }
    for (int i = 0; i < m; ++i) if (where[i] == -1) return 1e9;
    return 1;
}
```

READ 23. Advanced DP Optimizations (Li-Chao & D&C)

Li-Chao handles dynamic line minima; divide-and-conquer DP needs monotone optimal split points.

```cpp
// Li-Chao Tree (minimum line query) on an integer domain [lo, hi).
struct LiChao {
    using ll = long long;
    static constexpr ll INF = 4'000'000'000'000'000'000LL;
    struct Line {
        ll m = 0, c = INF;
        ll get(ll x) const {
            __int128 v = (__int128)m * x + c;
            if (v > INF) return INF;
            if (v < -(__int128)INF) return -INF;
            return (ll)v;
        }
    };
    struct Node { Line f; Node* l = nullptr; Node* r = nullptr; explicit Node(Line f) : f(f) {} };
    ll lo, hi; Node* root = nullptr;
    LiChao(ll lo, ll hi) : lo(lo), hi(hi) {}
    ~LiChao() { clear(root); }
    void add_line(Line nw) { add_line(root, lo, hi, nw); }
    ll query(ll x) const { return query(root, lo, hi, x); }
private:
    void clear(Node* p) { if (!p) return; clear(p->l); clear(p->r); delete p; }
    void add_line(Node*& p, ll l, ll r, Line nw) {
        if (!p) { p = new Node(nw); return; }
        ll m = l + (r - l) / 2;
        bool lef = nw.get(l) < p->f.get(l);
        bool mid = nw.get(m) < p->f.get(m);
        if (mid) swap(nw, p->f);
        if (r - l == 1) return;
        if (lef != mid) add_line(p->l, l, m, nw);
        else add_line(p->r, m, r, nw);
    }
    ll query(Node* p, ll l, ll r, ll x) const {
```

```
        if (!p) return INF;
        ll cur = p->f.get(x);
        if (r - l == 1) return cur;
        ll m = l + (r - l) / 2;
        return min(cur, x < m ? query(p->l, l, m, x) : query(p->r, m, r, x));
    }
};

// Generic divide-and-conquer DP optimization skeleton. Requires monotone optimal split points.
template<class Cost>
void dac_opt(int i, int l, int r, int ql, int qr, const vector<vector<long long>>& dp, vector<long long>&
 ↪  cur, Cost cost) {
    if (l > r || ql > qr) return;
    int mid = (l + r) / 2, bk = -1;
    long long best = LLONG_MAX / 4;
    for (int k = max(1, ql); k <= min(mid, qr); ++k) {
        long long v = dp[i - 1][k - 1] + cost(k, mid);
        if (v < best) best = v, bk = k;
    }
    cur[mid] = best;
    dac_opt(i, l, mid - 1, ql, bk, dp, cur, cost);
    dac_opt(i, mid + 1, r, bk, qr, dp, cur, cost);
}
```

READ 24. Digit DP - canonical count-by-digit-sum

```
// State = position, accumulated digit sum, and tight flag; memo size follows the input instead of hard-coding
 ↪  20 x 172.
long long count_digit_sum(const string& s, int target) {
    int n = (int)s.size();
    if (target < 0 || target > 9 * n) return 0;
    vector<vector<long long>> memo(n, vector<long long>(target + 1, -1));
    function<long long(int,int,bool)> dfs = [&](int pos, int sum, bool tight) -> long long {
        if (sum > target) return 0;
        if (pos == n) return sum == target;
        if (!tight && memo[pos][sum] != -1) return memo[pos][sum];
        int lim = tight ? s[pos] - '0' : 9;
        long long ans = 0;
        for (int d = 0; d <= lim; ++d)
            ans += dfs(pos + 1, sum + d, tight && d == (s[pos] - '0'));
        if (!tight) memo[pos][sum] = ans;
        return ans;
    };
    return dfs(0, 0, true);
}
```

READ 25. Aho-Corasick - complete multi-pattern matcher

```
// Build a trie plus failure links; each text character advances in O(1) amortized time.
struct AhoCorasick {
    struct Node { array<int,26> next{}; int link=0, out=0; Node(){next.fill(-1);} };
    vector<Node> t{Node()};
    void add(const string& s){
        int v=0;
        for(char ch:s){int
         ↪  c=ch-'a';if(t[v].next[c]==-1){t[v].next[c]=t.size();t.push_back(Node());}v=t[v].next[c];}
        ++t[v].out;
    }
    void build(){
        queue<int> q;
        for(int c=0;c<26;++c){int u=t[0].next[c];if(u==-1)t[0].next[c]=0;else q.push(u);}
        while(!q.empty()){int v=q.front();q.pop();t[v].out+=t[t[v].link].out;for(int c=0;c<26;++c){int
         ↪  u=t[v].next[c];if(u==-1)t[v].next[c]=t[t[v].link].next[c];else
         ↪  t[u].link=t[t[v].link].next[c],q.push(u);}}
```

```
    }
    long long matches(const string& s){long long ans=0;int v=0;for(char
     ↪  ch:s)v=t[v].next[ch-'a'],ans+=t[v].out;return ans;}
};
```

READ **26. Suffix Automaton - compact reference**

```
// Each state represents an end-position equivalence class; suffix links connect to the largest proper suffix
 ↪  class.
struct SuffixAutomaton {
    struct State { int next[26]; int link=-1,len=0; State(){fill(next,next+26,-1);} };
    vector<State> st; int last=0;
    SuffixAutomaton(const string& s){st.reserve(2*s.size());st.push_back(State());for(char c:s)extend(c-'a');}
    void extend(int c){
        int cur=st.size();st.push_back(State());st[cur].len=st[last].len+1;int p=last;
        while(p!=-1&&st[p].next[c]==-1)st[p].next[c]=cur,p=st[p].link;
        if(p==-1)st[cur].link=0;else{int q=st[p].next[c];if(st[p].len+1==st[q].len)st[cur].link=q;else{int
         ↪  clone=st.size();st.push_back(st[q]);st[clone].len=st[p].len+1;while(p!=-1&&st[p].next[c]==q)st[p
         ↪ ].next[c]=clone,p=st[p].link;st[q].link=st[cur].link=clone;}}
        last=cur;
    }
};
```

Sunday operating rule

The advanced vault is a reference bank, not a new weekly syllabus. Use it only when an OA/company style or a hard problem clearly demands it. The normal meal-reading list is the day sections above.